MR222768_desafío03

Sistema de pedidos con ASP.NET Core Web API

Arquitectura de N capas, Entity Framework Core, SQL Server, Docker y pruebas xUnit

Informe técnico con evidencias verificadas el 16 de octubre de 2026

DOCUMENTO PROVISIONAL: la implementación y las evidencias son reales; falta incorporar la URL definitiva de GitHub y, si corresponde, los datos institucionales de portada.

1. Resumen ejecutivo

El proyecto implementa una API REST para clientes, productos y pedidos. Incluye persistencia real en SQL Server mediante Entity Framework Core Code First, validaciones con Data Annotations, documentación OpenAPI, ejecución mediante Docker Compose y pruebas unitarias independientes de la base de datos.

Componente	Resultado verificado
Docker Desktop	4.93.0 en ejecución
Docker Engine	29.8.1, Linux/amd64 sobre WSL2
SQL Server	2022, contenedor saludable
API	ASP.NET Core 8 en http://localhost:8080
Compilación	0 errores, 0 advertencias
xUnit	19 aprobadas, 0 fallidas, 0 omitidas
Migración	20261017042631_InitialCreate aplicada

2. Arquitectura

La solución se divide en cuatro capas con dependencias dirigidas hacia el dominio y la aplicación. Los controladores no acceden directamente al DbContext y la lógica de inventario reside en servicios de aplicación.

Capa	Responsabilidad
SistemaPedidos.Domain	Entidades, validaciones fundamentales y excepciones de dominio.
SistemaPedidos.Application	DTOs, contratos de repositorios, servicios y reglas de negocio.
SistemaPedidos.Infrastructure	DbContext, configuraciones EF, repositorios, transacciones y migraciones.
SistemaPedidos.API	Controladores REST, Swagger, inyección de dependencias y manejo de errores.
SistemaPedidos.Tests	Pruebas xUnit con dobles de repositorio; no requieren SQL Server.

3. Modelo de datos

Entidad	Campos
Cliente	Id, Nombre, Apellido, Email
Producto	Id, Nombre, Descripcion, Precio decimal(18,2), Stock, Version rowversion
Pedido	Id, FechaPedido, Cantidad, ClienteId, ProductoId

Las relaciones Cliente-Pedido y Producto-Pedido usan claves foráneas con borrado restringido. La migración agrega índices para las claves foráneas, un índice único de email y restricciones CHECK para precio, stock y cantidad.

4. API REST

Recurso	Métodos	Ruta
Clientes	GET, POST	/api/clientes
Clientes	GET, PUT, DELETE	/api/clientes/{id}
Productos	GET, POST	/api/productos
Productos	GET, PUT, DELETE	/api/productos/{id}
Pedidos	GET, POST	/api/pedidos
Pedidos	GET, PUT, DELETE	/api/pedidos/{id}

Códigos HTTP utilizados

200 para consultas y actualizaciones; 201 para creaciones; 204 para eliminaciones; 400 para validaciones; 404 para registros inexistentes; 409 para conflictos de negocio o concurrencia; y 500 solo para fallos inesperados sin detalles internos.

5. Reglas de negocio y concurrencia

• No se crean pedidos para clientes o productos inexistentes.

• La cantidad y el precio deben ser mayores que cero; el stock no puede ser negativo.

• Un pedido se rechaza cuando el inventario es insuficiente.

• Crear, actualizar o eliminar pedidos ajusta automáticamente el stock.

• Cambiar el producto de un pedido restaura el inventario anterior y descuenta el nuevo.

• No se eliminan clientes ni productos con pedidos asociados.

• Los cambios del pedido y del inventario se guardan en una transacción Serializable.

• Producto.Version usa rowversion para detectar concurrencia y evitar sobreventa.

6. Pruebas automatizadas

La ejecución real de xUnit finalizó con 19 pruebas aprobadas, 0 fallidas y 0 omitidas. Las pruebas cubren validación por Data Annotations, creación de entidades, consultas existentes e inexistentes, stock insuficiente, descuento y restauración de inventario, actualización de cantidades, cambio de producto y restricciones de eliminación.

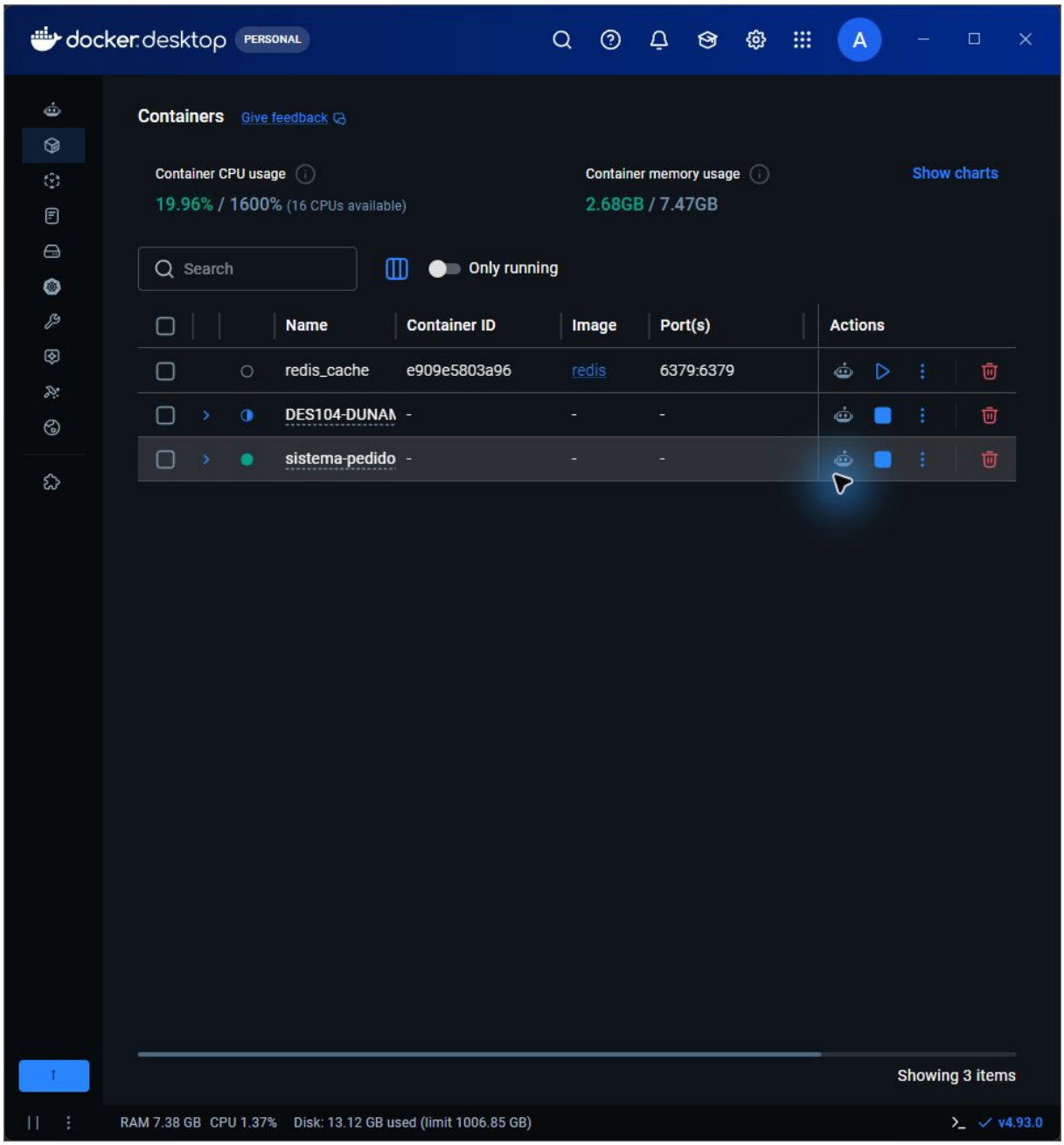
Resultado xUnit	19 aprobadas
Fallidas	0
Omitidas	0
Compilación	0 errores / 0 advertencias

7. Pruebas HTTP de integración

Caso	Resultado
Swagger/OpenAPI	200
Crear / consultar / actualizar cliente	201 / 200 / 200
Crear / consultar / actualizar producto	201 / 200 / 200
Crear / consultar pedido	201 / 200
Stock después de pedido de 3	12 a 9
Datos inválidos	400
Stock insuficiente	409
Registro inexistente	404
Eliminar cliente/producto asociado	409 / 409
Actualizar pedido a 5 unidades	200; stock a 7
Eliminar pedido	204; stock restaurado a 12
Limpieza de datos de prueba	204 / 204

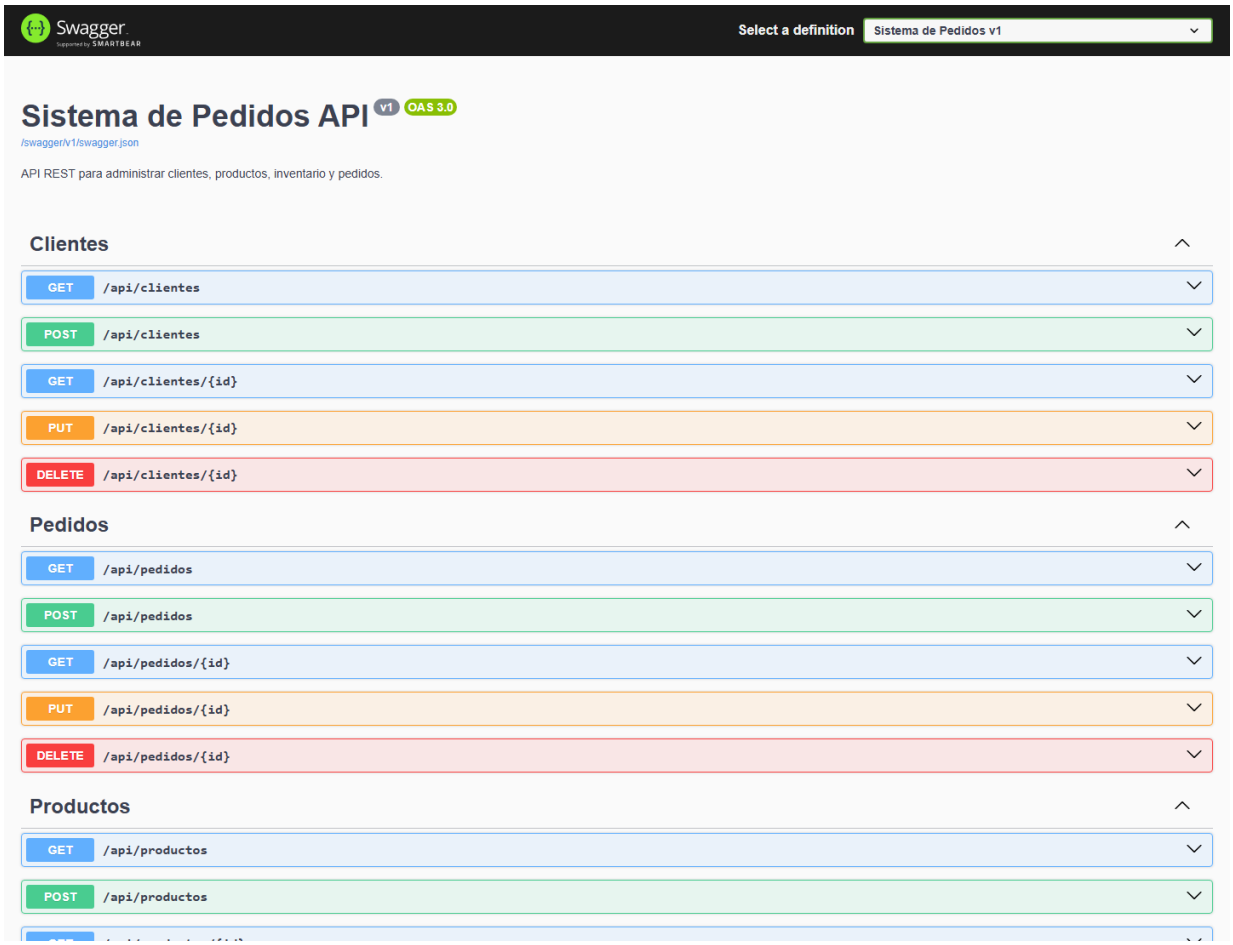
8. Evidencia visual: Docker

Captura real de Docker Desktop. El grupo sistema-pedidos se encuentra activo; la verificación por CLI confirmó ambos contenedores y el estado saludable de SQL Server.



9. Evidencia visual: Swagger

Captura real de la interfaz OpenAPI servida por la API en http://localhost:8080/swagger. Se observan los controladores de Clientes, Pedidos y Productos con sus operaciones REST.



10. Ejecución y despliegue

Docker Compose configura una red interna, un volumen persistente, SQL Server 2022 y la API. La conexión utiliza el nombre del servicio sqlserver y la contraseña se mantiene exclusivamente en el archivo local .env excluido por Git. El healthcheck espera a SQL Server y la API aplica las migraciones con reintentos al iniciar.

11. Conclusiones

La solución cumple el alcance solicitado y fue verificada de extremo a extremo. La separación por capas mantiene la lógica de negocio fuera de los controladores, EF Core proporciona persistencia y migraciones reproducibles, y la combinación de transacciones serializables con rowversion protege el inventario frente a operaciones concurrentes. Las pruebas unitarias y HTTP confirmaron las reglas principales y los códigos de respuesta.

12. Información pendiente

URL del repositorio GitHub	_____
Datos institucionales de portada	_____

Anexos disponibles en docs/evidencias/: salidas completas de restore, build y test; resultados HTTP en JSON; migración consultada en SQL Server; logs de la API; capturas y especificación OpenAPI.